

矩阵变换 (matrix)

首先发现, $(1, k_1)$ 接着 $(1, k_2)$ 等价于 $(1, k_1 + k_2)$; 然后我们会发现, 比如 $(1, k), (0, k), (1, k)$ 其实就等价于 $(1, k)$, 更一般地, 如果有 $a \geq b \geq c$, 那么 $(1, a), (0, b), (1, c)$ 就等价于 $(1, a), (0, b - c)$ 。 $a \geq b, b < c$ 的情况, $(1, a), (0, b), (1, c)$ 就等价于 $(1, a + c - b)$ 。

那么使用一个单调栈, 维护操作序列, 新加入操作的时候, 如果和栈顶类型相同就直接合并; 否则设栈顶元素是 $(1, a)$, 栈顶下面一个元素是 $(0, b)$, 如果 $a \geq b$, 就可以执行合并操作。容易发现, 这样保证了栈顶以外的所有元素是严格递增的, 而元素大小超过 n 就可以直接停止操作了 (整个棋盘都会被同一种颜色占领), 因此最终栈内元素个数是 $O(n)$ 的。其他也有一些方法做到 $O(n)$ 次操作, 然后使用 $O(qn^2)$ 的做法即可通过。

完美匹配 (match)

$k = 1$ 的情况就是求一下最大匹配, 用匈牙利算法就行。

$k = 2$ 的情况也是差不多的, 但需要一些讨论, 如果要做到快速判断 (不需要枚举改变颜色的边), 可能需要使用退流操作。

对于一般的情况, 先不考虑可以改变至多一条边的颜色的条件, 那么要求颜色集合恰好为 S 是否存在合法的方案, 考虑容斥, 因为只有颜色集合包含于 S 的时候, 处理起来才会方便。但是容斥需要我们找到一个可以加减的量, 而行列式正好满足了这一特性。

具体地, 设 $w_{i,j}$ 表示一个 $[1, p - 1]$ 之间的随机数, 对于 S , 设

$$a_{i,j} = \begin{cases} w_{i,j}, & (i, j, c) \in E \wedge c \in S \\ 0, & \text{otherwise} \end{cases}$$

此时令矩阵 $A = (a_{i,j})_{n \times n}$, 求出 $\det(A)$ 就是正好合适的量:

- 如果不存在完美匹配, 那么 $\det(A)$ 一定为 0;
- 如果存在完美匹配, 那么 $\det(A)$ 极大概率不为 0;
- 由行列式的定义:

$$\det(A) = \sum_p (-1)^{\text{sign}(p)} \prod_{i=1}^n a_{i,p_i}$$

可以发现, 一次求行列式就会求出所有完美匹配对应的权值和, 那么再对于颜色集合容斥, 根据答案是否为 0 判断有没有完美匹配即可。

剩下的问题，只需要解决可以修改一条边的情况，此时，我们只需要把矩阵中的元素修改为一个二元组 (x, y) ， x 表示不修改的方案的对应该权值， y 表示修改过一条边的方案对应的权值。二元组相加减就是 x, y 分别相加减，二元组相乘就是 $(x_1x_2, x_1y_2 + x_2y_1)$ 。本质上是在 $\text{mod } x^2$ 意义下的多项式加法和乘法。

这样处理之后，再进行容斥，就能够解决问题了。时间复杂度 $O(2^kn^3)$ 。

序列规划 (sequence)

首先，我们可以发现，所有未确定的 a_i ，只需要在已确定的 a_i 中选取即可，可以通过调整法得到这个结论。

因此值域其实相当于是 $n - k$ 的，所以就有一个 $O((n - k)^k \text{poly}(n))$ 的暴力，可以通过 $n \leq 10$ 。

接下来考虑正解，由于 $-\min\{a, x\}$ 和 $\max\{a, x\}$ 都是下凸函数，因此贡献也是凸的。

对于 a 中所有未知的位置集合 S 考虑，要么填 x ，要么填 $x + 1$ 。此时求出当前的最优解，那么一定有，填 x 的元素，在最终的最优解中一定 $\leq x$ ，填 $x + 1$ 的元素在最优解中一定 $\geq x + 1$ 。证明可以参考官方题解 ([H. Absolutely Flat](#))

Problem H. Absolutely Flat

Let b be an array of nonnegative integers. Fix a family of q segments, and define $f(b)$ to be the sum of oscillations of b on those segments.

For an array b and a nonnegative integer x , define another array b^x , such that $b_i^x = \begin{cases} 0, & b_i \leq x \\ 1, & b_i > x \end{cases}$. It is

easy to see that $f(b) = \sum_{x=0}^{\infty} f(b^x)$.

Now suppose that a is an array of nonnegative integers, where some elements are missing (we denote them below by $?$). Define $g(a)$ to be the minimum of f over all arrays of nonnegative integers b that match a .

It is easy to see that $g(a) \geq \sum_{x=0}^{\infty} g(a^x)$, where a^x consists of 0, 1, and $?$. In fact, there holds an equality (that is, the 01-principle works).

Proof

For two arrays c and d , let's write $c \leq d$ and $\min(c, d)$ in a componentwise meaning.

Let b_x be an optimal array that matches a^x (that is, b_x consists of zeroes and ones, matches a^x , and $f(b_x)$ is minimized). Define b_{x+1} analogously.

For any two arrays c and d of zeroes and ones, there holds an inequality $f(c) + f(d) \geq f(\min(c, d)) + f(\max(c, d))$ (which can be easily verified by considering $q = 1$). Therefore, $f(b_x) + f(b_{x+1}) \geq f(\min(b_x, b_{x+1})) + f(\max(b_x, b_{x+1}))$. On the other hand, b_x and b_{x+1} were the optimal arrays, and $\min(b_x, b_{x+1})$ also matches a^x . Hence $f(b_x) = f(\min(b_x, b_{x+1}))$ and $f(b_{x+1}) = f(\max(b_x, b_{x+1}))$. Hence one can replace b_{x+1} with $\min(b_x, b_{x+1})$. By induction, it could be assumed that $b_0 \geq b_1 \geq b_2 \geq \dots$, which is what we wanted.

Proof ends

题解中也介绍了 $O(n^2 \log n)$ 的做法，我们这里直接介绍正解。

具体地，把每个线段的贡献拆成两个，一个是 $\max\{a, x\}$ ，一个是 $-\min\{a, x\}$ ，然后设 $\text{solve}(S, T, L, R)$ 表示现在未知的元素集合为 S ，贡献集合为 T ，未知元素的值域一定在 $[L, R]$ 中。然后设 $x = \frac{L+R}{2}$ ，求出 S 中元素在 $\{x, x+1\}$ 中选择时的最优解。可以发现，此时所有的贡献的线段也一定只会对 $\leq x$ 和 $\geq x+1$ 中的一边产生影响，所以继续递归两边的子问题直到 $L = R$ 为止。

另外，如何求出 $\{x, x+1\}$ 中选择的最优解，这里可以使用 dp 加线段树解决。

这个方法实际上叫做保序回归（整体二分）解决该问题了。时间复杂度 $O((n+q) \log^2 n)$ ，足以通过。如果是 $O((n+q) \log n \log V)$ 的话，时间会比较大。

如果把 dp 加线段树的方法替换为单调栈加并查集的话，可以做到 $O((n+q)\alpha(n) \log n)$ 。